

Inyección de comandos de sistema operativo

Objetivos de aprendizaje

Distinguir shell de ejecución directa, predecir el significado de metacaracteres, construir pruebas in-band y blind con controles y adaptar quoting o canal sin probar separadores al azar.

Prerrequisitos

Procesos, argv, Bash, quoting, JSON, redirecciones y medición temporal.

Fundamentos técnicos

Command injection requiere que entrada controlada altere una orden interpretada por una shell u otro intérprete de comandos. En ejecución directa, el kernel recibe un ejecutable y una lista de argumentos: `;`, `&&` o `|` no tienen gramática propia y pueden llegar como texto al programa.

```
sh -c "ping -c 1 <entrada>"      -> la shell analiza operadores
execve("ping", ["ping", "-c", "1", entrada]) -> ping recibe un argumento
```

Modelo mental

```
HTTP/JSON -> valor -> concatenación/quoting -> ¿shell? -> expansiones/redirecciones
\----> ¿argv? -> parser de opciones
```

Superficie de ataque

Diagnósticos de red, conversión de ficheros, backups, búsquedas, integraciones y tareas que invocan utilidades. La presencia de un proceso externo no implica shell.

Cómo identificarla

- Un marcador de segundo comando aparece fuera de la salida normal.
- Una demora proporcional se reproduce frente a controles intercalados.
- Un callback OOB contiene token único generado por la orden.
- Los errores cambian según quoting u operador de una shell identificada.

Preguntas que debo hacerme

1. ¿Existe shell o ejecución directa?
2. ¿Dentro de qué comillas se inserta el valor?
3. ¿Qué operadores entiende esa shell?
4. ¿Se captura stdout, stderr o ningún canal?
5. ¿Qué identidad, entorno y directorio tiene el proceso?

Prueba mínima

Con salida visible, producir un marcador fijo mediante una orden inocua. Sin salida, usar una demora corta y dos duraciones con varias muestras, o un callback al listener autorizado. Siempre mantener petición original y una cadena literal como controles.

Construcción progresiva del payload

Para `sh -c "ping -c 1 $host"`: conservar un host válido, determinar quoting, añadir un separador, emitir MARK y predecir dónde aparece. Para `execve`, esa misma cadena no es un payload de command injection: es un único operando de `ping`.

Anatomía de los payloads

- Contexto de entrada: JSON host.
- Sintaxis original: `sh -c "ping -c 1 <host>"`.
- Entrada controlada: final de la orden, sin quoting adicional.
- Transformaciones conocidas: JSON decode y concatenación.
- Parser final: `/bin/sh` confirmado.
- Sink: ejecución de orden.
- Primitiva: segundo comando que emite marcador.
- Payload mínimo: `127.0.0.1; printf CI_OK`.
- Significado de cada componente: host válido; `;` termina la primera orden; `printf` emite marcador.
- Resultado esperado: salida de ping seguida de `CI_OK`.
- Control negativo: `127.0.0.1; printf CI_NO` enviado a una ruta que usa `execve`.
- Restricción observada: `stdout` no se devuelve en producción del laboratorio.
- Por qué falla la variante básica: la orden se ejecuta, pero el canal no es visible.
- Hipótesis de adaptación: usar efecto temporal proporcional.
- Payload adaptado: separador más una demora corta permitida en el laboratorio.
- Por qué debería funcionar: la shell espera al segundo proceso.
- Evidencia: distribución separada y proporcional para 2 s y 4 s.
- Cuándo no funcionaría: ejecución asíncrona, timeout, shell ausente o utilidad de demora bloqueada.

Variaciones según el contexto

`;` `id` puede: ejecutarse si una shell lo analiza sin quoting protector; aparecer literalmente si es un argumento; causar error si el programa valida el host completo; o no producir señal si la entrada se ignora o la salida no se captura.

Filtros y bypasses

Un carácter bloqueado no justifica probar todos los demás. Determina si el rechazo ocurre en JSON, validación, shell o programa. Una adaptación cambia quoting, operador o canal solo para probar una hipótesis concreta.

Evidencias de confirmación

Marcador, demora proporcional o callback atribuible a una segunda orden. Un error distinto al incluir `;` solo prueba que el carácter afectó alguna capa.

Escalado de impacto

Confirmar identidad y entorno con una capacidad mínima, evaluar lectura/escritura estrictamente necesaria y evitar saltar a una reverse shell si el canal existente basta para demostrar impacto.

Errores frecuentes

- Agrupar APIs con y sin shell como equivalentes.
- Confundir error de ping con error de shell.
- Usar una sola medida temporal.
- Escapar para la shell local, no para la remota.
- Interpretar fallo de reverse shell como refutación de ejecución.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
; id aparece en error de host	argv directo o quoting	observar argv; comparar expansión shell
; id cambia error, sin salida	shell parcial o validación	marcador temporal/OOB y stderr separado
demora consistente y proporcional	ejecución blind	alternar controles y dos duraciones
id funciona, conexión saliente no	egress/DNS/entorno	callback DNS/HTTP controlado y variables de red
; bloqueado antes del proceso	filtro textual	localizar fase; no enumerar separadores
JSON devuelve 400	transporte inválido	serializar objeto y verificar bytes

Mitigaciones

Usar APIs nativas o ejecución directa con lista fija de argumentos, validar valores semánticamente, eliminar shell, fijar entorno y privilegios, y registrar la invocación sin datos sensibles.

Relación con pentesting y certificaciones

La destreza evaluada es demostrar la gramática que procesa la entrada y elegir un canal robusto.

Caso guiado

Caso A — Básico: diagnóstico de host

La respuesta a `127.0.0.1; printf MARK` contiene salida de ping y MARK.

Observación: aparece un marcador de una segunda orden. Qué sé: alguna capa interpretó `;`. Hipótesis: shell en backend o interpretación inesperada anterior. Experimento: control literal, captura de comando del laboratorio y operador condicionado. Resultado: `log sh -c` y marcador solo con separador. Conclusión: command injection in-band. Siguiente paso: registrar UID y limitar impacto.

Caso de adaptación

Caso B — Sin salida visible

El mismo endpoint devuelve siempre `202 job queued`.

Observación: no hay canal in-band. Hipótesis: orden asíncrona, entrada no alcanzada o salida descartada. Experimento: demoras 2/4 s no sirven si la respuesta es asíncrona; se usa callback con token desde el worker del laboratorio y un control sin segundo comando. Resultado: el callback llega desde el worker solo con payload. Conclusión: ejecución blind confirmada; la ausencia de demora no la refutaba. Siguiente paso: documentar cola e identidad.

Caso C — Transferencia: nombre del archivo de backup

Una tarea web crea tar con un nombre de archivo proporcionado. El alumno observa que un nombre con `; printf MARK` genera un fichero adicional MARK en el directorio de pruebas.

Resolución: nombre -> cadena de shell -> tar -> segundo comando. Se confirma con un nombre literal escapado como control y con el comando construido. La función no se llamaba "ejecutar comando", pero la primitiva es idéntica.

Caso D — Falso positivo: ping recibe todo

`127.0.0.1; id` devuelve ping: `127.0.0.1; id: Name or service not known`.

Observación: el texto llegó a ping. Hipótesis: shell protegida o execve. Experimento: traza argv y expansión \$(printf X). Resultado: un único argumento literal. Conclusión: command injection descartada; revisar argument/option injection solo si puede crearse otro argumento.

Ejercicios

1. Predice el resultado de la misma entrada bajo sh -c y execve.
2. Diseña una medición temporal resistente a ruido.
3. Explica por qué una reverse shell fallida no refuta printf MARK exitoso.
4. Serializa por capas una cadena JSON que contenga comillas de shell.

Resumen

Los operadores solo son operadores para una shell. Primero se identifica parser y quoting; después se construye una primitiva y un canal causal.

Chuleta operativa

1. Cadena final o argv.
2. Shell y quoting.
3. Marcador mínimo.
4. Stdout, stderr, tiempo u OOB.
5. Control literal y repetición.
6. UID, entorno y directorio.

Referencias

- [PortSwigger: OS command injection](#)
- [GNU Bash Reference Manual](#)
- [Python subprocess](#)
- [Concepto interno](#) (command-injection)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Práctica: [cuaderno](#).